ChatGPT-like System using Gemini API with RAG & Docker

1. Abstract

This project is a ChatGPT-like conversational AI system built using modern backend technologies. The system allows users to interact with an intelligent chatbot capable of generating human-like responses, maintaining conversation memory, and retrieving relevant information using Retrieval-Augmented Generation (RAG).

The application integrates the Gemini API for advanced language understanding and generation. It also supports document-based querying using vector search techniques and provides an easy-to-deploy environment using Docker.

The main objective of this project is to simulate a real-world AI assistant system with scalable architecture and production-ready deployment practices.

🎯 2. Objectives

- Build a conversational AI system using LLMs
- Implement memory-based chat interactions
- Integrate RAG for knowledge retrieval
- Enable image/document-based querying (if implemented)
- Containerize the application using Docker
- Provide easy setup and execution for users

⚙ 3. Tech Stack

◇ Backend

- Python
- FastAPI

◇ AI & ML

- Gemini API (LLM)
- Sentence Transformers (for embeddings)
- FAISS (vector database for RAG)

◇ Dev Tools

- Git & GitHub
- Docker

🧩 4. System Architecture

The system consists of the following components:

1. **User Interface (Client Side)**
    a. Simple HTML frontend or API-based interaction
2. **FastAPI Backend**
    a. Handles API requests
    b. Manages chat logic and routing
3. **LLM Integration (Gemini API)**
    a. Generates responses
    b. Handles natural language understanding

4. **RAG Module**
    a. Retrieves relevant documents using embeddings
    b. Enhances response accuracy
5. **Vector Store (FAISS)**
    a. Stores document embeddings
    b. Enables semantic search
6. **Docker Container**
    a. Ensures consistent deployment

🔄 5. Workflow

1. User sends a query
2. Backend receives request via FastAPI
3. Query is processed:
    a. Check conversation memory
    b. Retrieve relevant data using RAG
4. Combined context is sent to Gemini API
5. Gemini generates response
6. Response is returned to user

🧱 6. Modules Description

◇ 1. Chat Module

- Handles user input and response generation
- Maintains conversation history

◇ 2. RAG Module

- Converts documents into embeddings
- Retrieves relevant context using FAISS

◇ 3. LLM Module

- Uses Gemini API for generating intelligent responses

◇ 4. API Module (FastAPI)

- Provides REST endpoints
- Handles request/response flow

◇ 5. Docker Module

- Containerizes the application
- Ensures platform-independent execution

📡 7. API Endpoints (Example)

Endpoint	Method	Description
`/chat`	POST	Send user query and get response
`/upload`	POST	Upload document for RAG
`/health`	GET	Check server status

🐳 8. Docker Setup

Build Image

```
docker build -t chatbot-app .
```

Run Container

```
docker run -p 8000:8000 chatbot-app
```

9. How to Run the Project

Local Setup

```
pip install -r requirements.txt
uvicorn main:app --reload
```

Docker Setup

```
docker build -t chatbot-app .
docker run -p 8000:8000 chatbot-app
```

10. Environment Variables

Create `.env` file:

```
GEMINI_API_KEY=your_api_key_here
```

11. Challenges Faced

- Handling LLM response consistency
- Managing conversation memory
- Integrating RAG efficiently
- Docker dependency issues

📈 12. Future Enhancements

- Add user authentication
- Improve UI (React frontend)
- Add real-time streaming responses
- Deploy on cloud (AWS / Render)
- Add multi-user chat sessions

▩ 13. Conclusion

This project demonstrates the integration of modern AI technologies with backend development practices. It provides a scalable and deployable solution for building intelligent conversational systems. The use of Docker ensures portability, while RAG enhances response accuracy, making the system more reliable and efficient.